

Herald



Herald — commons_prefix Module Guide (Operator / Developer)

Field	Value
Revision	1
Created	2026-05-31
Last modified	2026-05-31
Status	active
Status summary	Nano-detail per-module reference for <code>commons_prefix</code> — the spec V3/V4 §8.2 deterministic 3-letter project-prefix algorithm. Documents the two exported functions (<code>Generate</code> , <code>Resolve</code>), the <code>tokenize</code> →Rule-A/B/C→uppercase pipeline, the <code>fnv1a32</code> -based collision-resolution pass, and every edge case the code actually handles (empty name → HRD, no-consonant fallbacks, CamelCase + delimiter splitting). ANTI-BLUFF: every claim below is grounded in <code>commons_prefix/prefix.go</code> + <code>commons_prefix/prefix_test.go</code> as read this revision — the worked examples are the literal <code>TestGenerate</code> table.
Issues	(none specific to this guide)
Continuation	bump when the planned <code>.herald/prefix.lock</code> (TOML) read/write + atomic-commit caller lands (today <code>Resolve</code> takes the existing map as a parameter; the lock-file persistence layer is not yet in this module), and when full Unicode NFKD diacritic-folding replaces the current ASCII-retain approximation.

Table of contents

- [§1. Overview](#)
 - [§2. Where the prefix is used](#)
 - [§3. The API](#)
 - [§4. The algorithm, step by step](#)
 - [§5. Worked examples \(the real test table\)](#)
 - [§6. Collision resolution \(`Resolve`\)](#)
 - [§7. Edge cases the code handles](#)
 - [§8. Testing notes](#)
 - [§9. References](#)
-

§1. Overview

`commons_prefix` (Go package `prefix`, module path `github.com/vasic-digital/herald/commons_prefix`) implements the spec §8.2 algorithm that derives a deterministic **3-letter uppercase prefix** from an arbitrary project name. It is a small, self-contained, dependency-free module — the only imports are the Go standard library (`hash/fnv`, `regexp`, `strings`, `unicode`).

The motivating problem (spec §8.2 / R-17): when a project consuming Herald does **not** define its own workable-item prefix (Herald's own is the hard-coded `HRD-`), Herald must invent one from the project name — and that invented prefix must be **stable** across machines and regenerations. So the algorithm is purely deterministic: the same input name always yields the same prefix, with no randomness and no clock/environment dependence.

The package doc records the rationale for shipping a bespoke generator: no mature Go library generates 3-letter abbreviations from arbitrary names (the only Go prior art, `Defacto2/releaser/initialism`, is a curated lookup table, not a generator), so Herald ships its own ~80-LOC implementation.

Path note (spec drift worth knowing). Spec §8.2 says "Implementation lives in `commons/prefix`." The actual implementation is the standalone workspace module `commons_prefix/` (package `prefix`), not a sub-package of `commons/`. The behaviour matches the spec; only the path text in §8.2 is stale.

§2. Where the prefix is used

The 3-letter prefix is the anchor for **workable-item identifiers**. Herald itself uses `HRD-` (e.g. `HRD-042`); a consuming project without its own prefix gets a derived one from this module. Concretely the prefix feeds:

- **Workable-item IDs** — the `<PREFIX>-NNN` id space (spec §8.3 lifecycle): the derived prefix replaces `HRD-` when the consuming project hasn't set one. The project name is taken from `package.json` / `go.mod` / `pyproject.toml` / the git remote (spec §18.2).
- **Per-flavor branding anchors** — the spec carries a `Prefix` field on the `Branding` struct (`docs/specs/mvp/specification.V4.md` §793/§1490: `Prefix string // 3-letter §8.2 anchor` (e.g. `"PHR"`, `"SHR"`, `"CHR"`)), the 3-letter anchor used in flavor branding.
- **The `.herald/prefix.lock` mapping** — the collision-resolved name → prefix table the caller persists (TOML, committed) so the assignment is reproducible. This module produces the value; the lock-file read/write is the caller's responsibility (see §6).

Note this is distinct from the *flavor-binary* naming convention (`pherald`, `sherald`, ...), which is a fixed single-letter `<x>herald` scheme, not output of this algorithm. `Generate/Resolve` operate on whole project names, not flavor letters.

§3. The API

The package exports exactly two functions; everything else (`tokenize`, `splitCamel`, `firstASCII`, `firstInternalConsonant`, `lastConsonant`, `isConsonant`, `fnv1a32`) is unexported internal machinery.

§3.1 Generate

```
func Generate(name string) string
```

Returns the deterministic 3-letter uppercase prefix for name, applying the tokenize → Rule-A/B/C → uppercase pipeline of §4. Pure function — no collision awareness, no I/O. An empty/garbage name (no usable tokens) returns the literal fallback "HRD" (Herald's own prefix).

§3.2 Resolve

```
func Resolve(name string, existing map[string]string) string
```

Calls Generate, then applies a deterministic collision-resolution pass against existing (the current prefix → name map, i.e. the contents the caller loaded from .herald/prefix.lock). If the generated prefix is free, or already owned by the SAME name, it is returned unchanged; otherwise the third character is rewritten deterministically (see §6). The caller MUST treat the return value as authoritative and write it back to the lock file (atomic + committed) — the persistence itself is **not** done inside this module.

§4. The algorithm, step by step

Generate runs five steps (the package doc + spec §8.2 both spell these out; the code below is the source of truth).

§4.1 Tokenize (tokenize)

1. **Strip** — iterate runes; keep Unicode letters and digits, keep the four delimiter runes - `<space> /`, drop everything else. (This is a deliberate ASCII-leaning **approximation** of NFKD diacritic-folding — full ICU is not pulled in; operators with non-Latin names map them manually in the lock file. The package comment says so explicitly.)
2. **Split on delimiters** — regexp `[-_ /]+` splits the stripped string; empty parts are discarded.
3. **Split each part on CamelCase boundaries** (`splitCamel`) — a boundary is a lower→upper transition (`IsLower(prev) && IsUpper(curr)`). So `HeraldRouter` becomes `["Herald", "Router"]`.

The token count after this drives which rule fires.

§4.2 Rule A — ≥ 3 tokens

First ASCII letter/digit of each of the **first three** tokens (extra tokens are ignored).

`HeraldRouterCore` → `H R C` → `HRC`; `my_cool_test_project` (4 tokens) → `M C T` → `MCT`.

§4.3 Rule B — exactly 2 tokens

First letter of token 1; first letter of token 2; **first internal consonant** of token 2.

`HeraldRouter` → `H R` + first internal consonant of `Router` (`T`) → `HRT`. `HeraldRunner` → `H R` + first internal consonant of `Runner` (`N`) → `HRN`.

§4.4 Rule C — exactly 1 token

First letter; **first internal consonant**; **last consonant** of the single token. `Herald` → H + first internal consonant (R) + last consonant (D) → HRD. `Project` → P R T → PRT.

§4.5 Uppercase

All three rules wrap their three bytes in `strings.ToUpper`, so the result is always uppercase regardless of input casing.

§4.6 The consonant helpers

- `firstASCII(s)` — first ASCII letter/digit, uppercased; returns 'X' if the token has none (an intentional "no first letter" marker).
- `isConsonant(b)` — true for A–Z excluding the vowels A E I O U **and** Y (Y is treated as a vowel here).
- `firstInternalConsonant(s)` — first consonant at index ≥ 1 (skips the first rune); **falls back to** `firstASCII(s)` when no internal consonant exists.
- `lastConsonant(s)` — last consonant scanning from the end of the uppercased token; **falls back to** `firstASCII(s)` when the token has no consonant at all.

These fallbacks are what make degenerate tokens (single letters, all-vowel tokens) still produce a 3-character result — see §7.

§5. Worked examples (the real test table)

These are the literal cases in `TestGenerate / TestCamelCaseEdgeCases` (`commons_prefix/prefix_test.go`), so they are guaranteed accurate:

Input name	Rule	Output	Why
<code>Herald</code>	C (1 token)	HRD	H + first internal consonant R + last consonant D.
<code>Project</code>	C (1 token)	PRT	P + first internal consonant R + last consonant T.
<code>HeraldRouter</code>	B (2 tokens via CamelCase)	HRT	H + R + first internal consonant of Router = T.
<code>HeraldRunner</code>	B (2 tokens)	HRN	H + R + first internal consonant of Runner = N.
<code>HeraldRouterCore</code>	A (3 tokens)	HRC	first letter of each of the first three tokens.
<code>my-project</code>	B (2 tokens, hyphen)	MPR	M + P + first internal consonant of project = R.
<code>my_cool_test_project</code>	A (4 tokens → first 3)	MCT	M C T; the 4th token is ignored.
<code>foo/bar/baz</code>	A (3 tokens, slash)	FBB	first letter of foo, bar, baz.
<code>" "</code> (empty)	fallback	HRD	no tokens → Herald's own prefix.
<code>X</code>	C, all-fallback	XXX	X + (no internal consonant → <code>firstASCII=X</code>) + (no last consonant → <code>firstASCII=X</code>).

Input name	Rule	Output	Why
AB	C (1 token)	ABB	A + first internal consonant B + last consonant B.

§6. Collision resolution (Resolve)

`Resolve(name, existing)` guarantees a unique prefix against the supplied `existing` map (`prefix` → `owning-name`):

1. `base := Generate(name)`. If `existing[base]` is absent **or** already owned by this same name, return `base` unchanged.
2. Otherwise compute `hash := fnv1a32(name)` and iterate `i = 0..25`: set the **third** character to `'A' + (i + hash) % 26`, and return the first candidate that is free or self-owned. (Only the third character is rewritten — the first two are preserved; `TestResolveCollisionTieBreak` asserts exactly this.)
3. If all 26 letters collide, iterate the third character over digits `'0'.. '9'` (the `HR0...HR9` numeric fallback for, e.g., `Herald`).
4. If even those 36 candidates are exhausted (vanishingly unlikely), return the literal `"HR0"` and leave manual disambiguation to the operator via a lock-file edit.

Because the rewrite is driven by `fnv1a32(name)` (a fixed hash of the name) and a fixed iteration order, `Resolve` is **deterministic** — `TestResolveDeterministic` asserts two calls with the same arguments return the same prefix.

Operator guidance. `Resolve` does NOT read or write `.herald/prefix.lock` — it takes the already-loaded map and returns the resolved value. The caller is responsible for loading the lock file into `existing`, calling `Resolve`, and writing the result back atomically (committed, per spec §8.2 step 7). That persistence layer is planned but not part of this module today.

§7. Edge cases the code handles

- **Empty / all-stripped name** → `"HRD"` (Herald's own prefix). Any name that tokenizes to zero tokens hits this fallback.
- **Single-letter token (X)** → `"XXX"`: no internal consonant and no last consonant, so both consonant helpers fall back to `firstASCII`, which returns the letter itself.
- **All-vowel / no-consonant tokens** → consonant helpers fall back to `firstASCII`, so the result is still 3 characters (never shorter, never panics).
- **Y is a vowel** — `isConsonant` excludes `Y`, so a name like `Yyy` would not treat `Y` as a consonant; the fallbacks then apply.
- **More than 3 tokens** → only the first three are used (Rule A); trailing tokens are dropped (`my_cool_test_project` → `MCT`).
- **Mixed delimiters + CamelCase** — `delimiter-split` runs first, then each part is `CamelCase-split`, so `foo/BarBaz` would yield `["foo", "Bar", "Baz"]`.
- **Non-ASCII letters** — kept through the strip step (they are Unicode letters) but `firstASCII` only emits ASCII; a token whose first rune is non-ASCII falls through to the `'X'` marker. This is the documented approximation, not full ICU folding.

§8. Testing notes

Tests live in `commons_prefix/prefix_test.go` and run with no external services or fixtures (pure functions):

```
go test -race -count=1 ./commons_prefix/...
```

(Verified PASS this revision: ok github.com/vasic-digital/herald/commons_prefix.)

Test	Proves
TestGenerate	The full Rule-A/B/C table of §5 — CamelCase, hyphen, underscore, slash, and the empty→HRD fallback.
TestResolveNoCollision	Resolve against an empty map returns the bare Generate result (HRD).
TestResolveSameOwnerReturnsExisting	A prefix already owned by the SAME name is returned unchanged (idempotent re-resolution).
TestResolveCollisionTieBreak	A clash with a DIFFERENT owner rewrites only the third letter (first two preserved), still 3 letters.
TestResolveDeterministic	Two Resolve calls with identical arguments return the identical prefix (no randomness).
TestCamelCaseEdgeCases	The degenerate-token fallbacks: ""→HRD, X→XXX, AB→ABB.

Anti-bluff observation worth preserving when editing tests:

TestResolveCollisionTieBreak deliberately asserts `got[0] == 'H' && got[1] == 'R'` to lock in the "only the third character changes" contract — keep that assertion if you touch the collision logic, since it is the only guard against an accidental full-prefix rewrite.

§9. References

- Source: `commons_prefix/prefix.go` and `commons_prefix/prefix_test.go`.
- Package doc: the comment block at the top of `prefix.go` (the §8.2 restatement + the "no mature Go library" rationale).
- Spec: `docs/specs/mvp/specification.V4.md` §8.2 "Derived 3-letter prefix algorithm" (the authoritative algorithm + collision-resolution + lock-file persistence rules); §8.3 (the HRD-NNN workable-item lifecycle the prefix anchors); §18.2 (project-name source resolution).
- Module: `commons_prefix/go.mod` (module `github.com/vasic-digital/herald/commons_prefix`, Go 1.22, zero third-party dependencies).

Sources verified

This guide documents internal Herald source only; no external service/library online documentation was relied on. All behavioural claims are grounded in the cited source files.

Verified 2026-05-31: internal doc — no external online sources. Behavioural claims derive from `commons_prefix/prefix.go` + `commons_prefix/prefix_test.go` + the §8.2/§8.3/§18.2 sections of `docs/specs/mvp/specification.V4.md` (all read 2026-05-31); the module has zero third-party dependencies (`commons_prefix/go.mod`), so no online-doc

cross-reference is required. Re-verify on any change to the Generate/Resolve API or the §8.2 spec text.